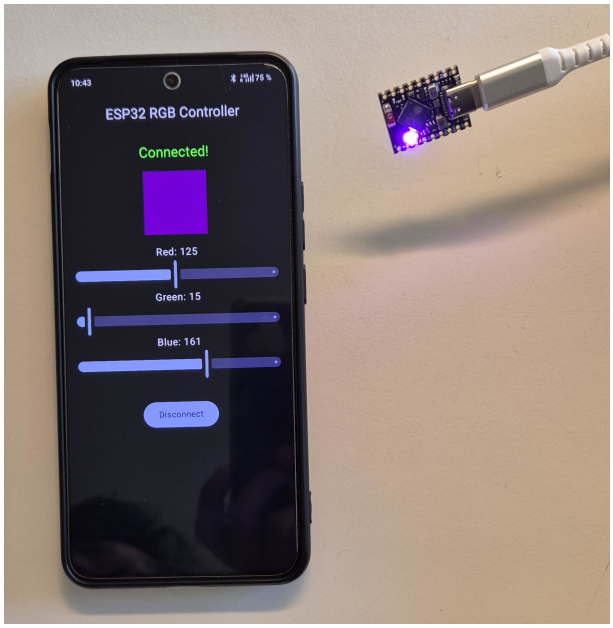
ESP32-S3 Secure Embedded System

Technical Report

Security Architecture (Flash Encryption, NVS Encryption, Secure Boot V2),
Secure OTA Firmware Updates,
and BLE-Controlled RGB LED with Android Companion App



MUSUMECI Lucas

Internship Project

April 2026

Git: https://github.com/lucasmusumeci/ESP32-Secure-OTA-BLE-Project

Abstract

This report documents the design and implementation of a secure embedded system built on the ESP32-S3 microcontroller using the ESP-IDF framework. The project combines three interconnected technical domains. First, a layered hardware and software security architecture is described, covering Flash Encryption (AES-XTS, hardware level), NVS Encryption (AES-XTS, software level), and Secure Boot V2 (RSA-3072 signature chain). Second, a complete Over-the-Air (OTA) firmware update system is presented, including a custom Python HTTPS server with range-request support, semantic version comparison, and dual-partition rollback protection. Third, a Bluetooth Low Energy (BLE) subsystem exposes a GATT service on the ESP32-S3 to receive RGB colour commands from a Jetpack Compose Android application. Together these elements form a production-grade demonstration of secure device management for IoT applications.

Contents

1 Introduction **3**
1.1 Project Architecture Overview . . . 3

2 Security Architecture on ESP32-S3 **4**
2.1 Overview of the Security Stack . . . 4
2.2 Flash Encryption . . . 4
2.2.1 Purpose and Threat Model . . . 4
2.2.2 Encryption Mechanism . . . 4
2.2.3 eFuses and the Security Model . . . 5
2.2.4 Partition-Level Behaviour . . . 5
2.2.5 Runtime API Behaviour . . . 5
2.2.6 Development vs. Release Mode . . . 6
2.3 NVS Encryption . . . 6
2.3.1 Why Flash Encryption Alone is Insufficient . . . 6
2.3.2 The NVS Storage Format . . . 6
2.3.3 Encryption Scheme . . . 6
2.3.4 The NVS Key Partition . . . 6
2.3.5 Initialisation Flow and Implementation . . . 7
2.3.6 NVS Encryption vs. Flash Encryption: Comparison . . . 8
2.4 Secure Boot V2 . . . 9
2.4.1 Purpose and Threat Model . . . 9
2.4.2 How Secure Boot V2 Works . . . 9
2.4.3 Key Generation . . . 9
2.4.4 Manual Signing Workflow for Fleet Management . . . 10
2.4.5 Interaction with OTA Updates . . . 10

3 Over-the-Air (OTA) Firmware Updates **11**
3.1 System Overview . . . 11
3.2 Firmware Server . . . 11
3.2.1 TLS Certificate . . . 11
3.2.2 Range-Request Support . . . 12
3.3 ESP32-S3 Firmware Side . . . 13

3.3.1 Partition Table . . . 13
3.3.2 CMakeLists.txt and Certificate Embedding . . . 13
3.3.3 OTA Configuration Structures . . . 14
3.3.4 Semantic Version Check and Rollback Prevention . . . 14
3.3.5 Download Loop and Finalisation . . . 15
3.3.6 Rollback Protection . . . 15

4 Bluetooth LE Subsystem **17**
4.1 Overview . . . 17
4.2 Firmware — NimBLE GATT Server . . . 17
4.2.1 Service and Characteristic Definitions . . . 17
4.2.2 GATT Write Handler . . . 18
4.2.3 GAP Advertising and Reconnection . . . 18
4.2.4 LED Driver . . . 19
4.3 Android Application — Jetpack Compose . . . 19
4.3.1 Permissions and Manifest . . . 19
4.3.2 BLE Manager . . . 20
4.3.3 UI — Jetpack Compose . . . 20

5 Conclusion **22**

1 Introduction

Modern IoT devices face a range of threats: physical access attacks that attempt to read or replace firmware, network-level attacks targeting update channels, and software vulnerabilities that could expose stored credentials. This project addresses these threats through the integration of three complementary security layers — Flash Encryption, NVS Encryption, and Secure Boot V2 — into a real embedded application that controls a WS2812B addressable RGB LED via Bluetooth, while also supporting authenticated over-the-air firmware updates.

The hardware platform is the **ESP32-S3**, a dual-core Xtensa LX7 microcontroller produced by Espressif Systems, featuring hardware accelerators for AES, SHA, and RSA, as well as eFuse-based key storage. The firmware is developed in C using the **ESP-IDF framework** (v6.x). The companion Android application is written in Kotlin using **Jetpack Compose**.

1.1 Project Architecture Overview

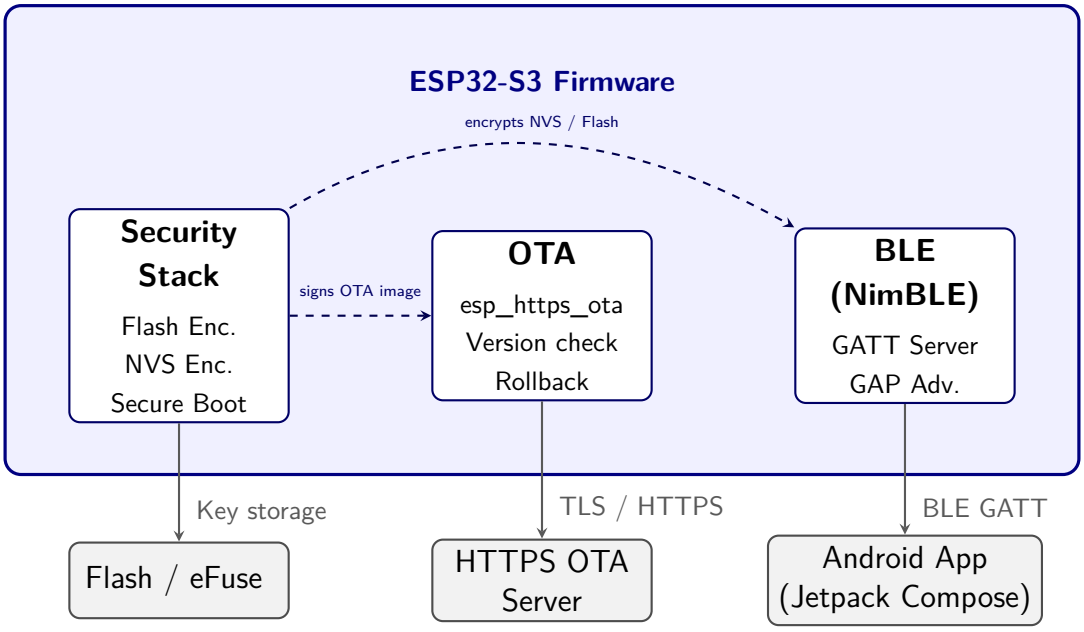


Figure 1: Firmware subsystems and their external interfaces

The ESP32-S3 firmware runs three concurrent FreeRTOS tasks: a NimBLE host task handling BLE communication, an OTA update task triggered after WiFi connection, and the main application task managing LED output. The security stack underpins all three: NVS encryption protects stored configuration, flash encryption protects all persistent code and data, and Secure Boot ensures only signed images execute.

2 Security Architecture on ESP32-S3

2.1 Overview of the Security Stack

The ESP32-S3 security model is best understood as a stack of independent but complementary layers, each addressing a different threat and a different level of the system. Table 1 summarises the three layers implemented in this project.

Table 1: Security layers implemented in the project

Layer	Mechanism	Primary Threat Addressed
Secure Boot V2	RSA-3072 signature verified by ROM	Malicious firmware replacement
Flash Encryption	AES-XTS hardware, eFuse-stored key	Physical flash chip reading/-cloning
NVS Encryption	AES-XTS software, key in `nvs_keys` partition	Code-execution attacks on stored data

Together these three mechanisms establish a **chain of trust**: the boot ROM verifies the bootloader (Secure Boot), the bootloader verifies the application image (Secure Boot), the flash hardware ensures raw bytes are unintelligible (Flash Encryption), and the NVS library ensures application-level key-value data is protected even from code running on the device (NVS Encryption).

2.2 Flash Encryption

2.2.1 Purpose and Threat Model

Without flash encryption, any attacker with physical access to the device and an off-the-shelf SPI flash programmer can read the entire firmware binary, extract credentials or API keys, and clone the device. Flash encryption eliminates this threat by ensuring that raw bytes read from the flash chip are ciphertext rather than intelligible data.

2.2.2 Encryption Mechanism

Flash encryption on the ESP32-S3 uses **AES-XTS** (XEX-based Tweaked-codebook mode with ciphertext Stealing), the same algorithm used by most full-disk encryption implementations (e.g. BitLocker, FileVault). A unique encryption key is generated on-device during first boot and stored in eFuses, where it is permanently locked. The key never leaves the chip.

Hardware Transparency

Encryption and decryption are performed entirely in hardware. Application code, the NVS library, and filesystems (FATFS, SPIFFS) read and write plaintext without any modification. Only `esp_flash_read()`, which bypasses the hardware cache, returns raw ciphertext.

2.2.3 eFuses and the Security Model

eFuses are one-time-programmable registers inside the chip. The encryption key is stored in eFuse key blocks and locked against read-back after first boot. The `FLASH_CRYPT_CNT` eFuse tracks the current encryption state: a value of 0 means encryption is disabled; a non-zero value means it is active. Because eFuses can only be burned once, this state is irreversible — making flash encryption a *hardware root of trust* rather than a software policy.

2.2.4 Partition-Level Behaviour

Flash encryption is not applied uniformly to every partition. The ESP32-S3 enforces mandatory encryption on critical system regions and leaves other regions under developer control via the `encrypted` flag in `partitions.csv`.

Table 2: Encryption behaviour by partition type

Partition Type	Flag Absent	Flag Present
App (`factory`, `ota_x`)	Encrypted (mandatory)	Encrypted
NVS Keys (`nvs_keys`)	Encrypted (mandatory)	Encrypted
OTA Data (`otadata`)	Encrypted (mandatory)	Encrypted
Custom Data (`0xff`, SPIFFS)	**Plaintext**	Encrypted

A common misconfiguration is to rely solely on mandatory hardware encryption without marking custom data partitions. Sensitive user data stored in, for example, a raw `0xff` partition would remain in plaintext unless the `encrypted` flag is explicitly set.

2.2.5 Runtime API Behaviour

```
// For an encrypted partition:
esp_partition_read(part, offset, buf, len);
// Returns: plaintext  (cache decrypts)
esp_flash_read(flash, buf, addr, len);
// Returns: ciphertext (bypasses cache)

// For a non-encrypted partition:
esp_partition_read(part, offset, buf, len);
// Returns: plaintext
esp_flash_read(flash, buf, addr, len);
// Returns: plaintext (same)
```

Listing 1: Observable difference between `esp_partition_read` and `esp_flash_read` when encryption is active

2.2.6 Development vs. Release Mode

Development vs. Release

Development mode allows re-flashing via `idf.py encrypted-flash` and keeps UART/JTAG access active. It is suitable for prototyping and testing.
Release mode permanently disables UART download and JTAG. It must be used for any production deployment. The difference is a single `menuconfig` toggle under *Security Features → Flash Encryption → Enable usage mode.*

Once flash encryption is active, standard `idf.py flash` must be replaced by:

```
idf.py encrypted-flash
```

Listing 2: Correct flashing command after encryption is enabled

This command pre-encrypts the binary before transmission, ensuring consistency with what the hardware expects.

2.3 NVS Encryption

2.3.1 Why Flash Encryption Alone is Insufficient

Flash encryption protects against offline attacks (reading the chip with a programmer). However, it provides no protection against an attacker who has already obtained *code execution* on the device: any running code can call `esp_partition_read()` and receive fully decrypted NVS data through the hardware cache. NVS encryption closes this gap.

2.3.2 The NVS Storage Format

The NVS library organises flash storage into 4096-byte pages. Each page starts with a 32-byte header, followed by a bitmap and a sequence of 32-byte entries, each storing one key–value pair. Without NVS encryption, entry data (Wi-Fi passwords, device certificates, calibration values) is written in plaintext.

2.3.3 Encryption Scheme

NVS encryption uses **AES-XTS** with a 256-bit key. Critically, only the *value (data) portion* of each 32-byte entry is encrypted — entry headers and metadata (key names, type fields, free-space bitmap) remain in plaintext so the NVS library can navigate the page structure without a full decryption pass.

The tweak used for AES-XTS is derived from the entry's flash address, which means two entries with identical plaintext values produce different ciphertext — preventing pattern analysis.

2.3.4 The NVS Key Partition

The AES-XTS encryption and tweak keys (`eky` and `tky`) are stored in a dedicated partition of type `data`, subtype `nvs_keys`. This partition also holds a CRC32 checksum over the two keys. The security of NVS encryption therefore reduces to the security of this partition, which is itself protected by flash encryption:

```
# Name,      Type,  SubType,  Offset,  Size,   Flags
nvs,         data,  nvs,      ,        0x6000,
nvs_key,     data,  nvs_keys, ,        0x1000, encrypted
ota_0,       app,   ota_0,    ,        1800K,
ota_1,       app,   ota_1,    ,        1800K,
```

Listing 3: Partition table extract — note the `encrypted` flag on `nvs_key` only

Critical: Never mark NVS partitions with the `encrypted` flag

If an NVS data partition were marked `encrypted`, flash encryption would re-encrypt the AES-XTS ciphertext already written by the NVS layer on every write, and re-decrypt it on every read, completely breaking the NVS encryption scheme. ESP-IDF detects this and returns `ESP_ERR_NVS_WRONG_ENCRYPTION`.

2.3.5 Initialisation Flow and Implementation

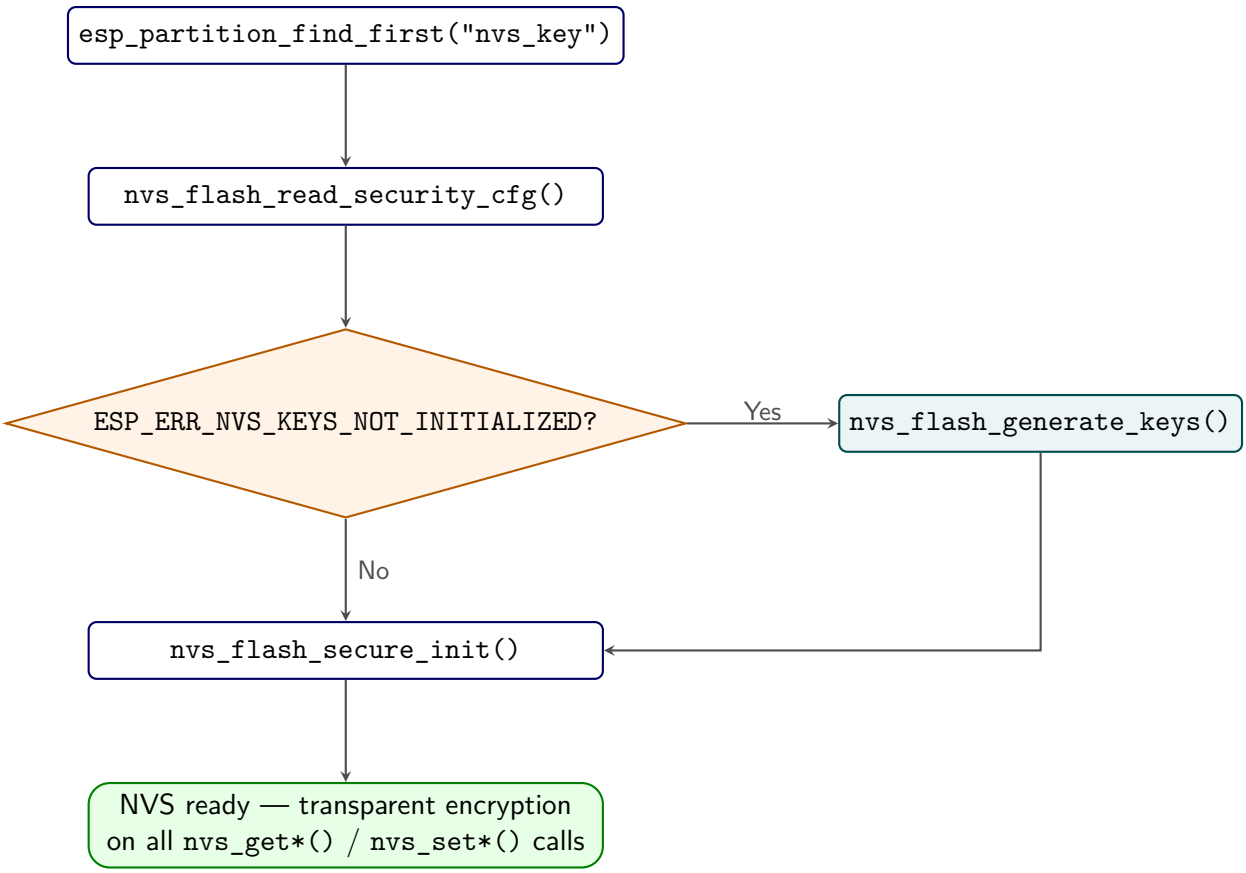


Figure 2: Boot-time NVS encryption initialisation sequence

```
esp_err_t init_encrypted_nvs() {
    // 1. Locate the nvs_key partition
    const esp_partition_t* key_part = esp_partition_find_first(
        ESP_PARTITION_TYPE_DATA,
        ESP_PARTITION_SUBTYPE_DATA_NVS_KEYS,
        "nvs_key"
```

```
    );
    if (key_part == NULL) {
        ESP_LOGE("NVS", "nvs_key partition not found!");
        return ESP_FAIL;
    }

    // 2. Read existing keys, or generate them on first boot
    nvs_sec_cfg_t cfg;
    esp_err_t err = nvs_flash_read_security_cfg(key_part, &cfg);
    if (err == ESP_ERR_NVS_KEYS_NOT_INITIALIZED) {
        ESP_ERROR_CHECK(nvs_flash_generate_keys(key_part, &cfg));
    } else {
        ESP_ERROR_CHECK(err);
    }

    // 3. Initialise the NVS partition with encryption
    err = nvs_flash_secure_init(&cfg);
    if (err == ESP_ERR_NVS_NO_FREE_PAGES
        || err == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        err = nvs_flash_secure_init(&cfg);
    }
    return err;
}
```

Listing 4: NVS encryption initialisation — `init_encrypted_nvs()` from `main.c`

Once `nvs_flash_secure_init()` has been called, all subsequent `nvs_get_*()` and `nvs_set_*()` calls transparently encrypt and decrypt data. No changes to application code are required.

2.3.6 NVS Encryption vs. Flash Encryption: Comparison

Table 3: Flash Encryption vs. NVS Encryption

	Flash Encryption	**NVS Encryption**
Layer	Hardware (AES engine in flash controller)	Software (AES-XTS in ESP-IDF)
Scope	Entire flash regions marked `encrypted`	NVS entry data only
Key storage	eFuse (hardware-burned, one-time)	`nvs_keys` partition
Key protection	Burned into eFuse, never extractable	Protected by flash encryption
Protects against	Physical flash reading/cloning	Code-execution, per-partition data exposure
API change needed	None	None (transparent after `secure_init`)

2.4 Secure Boot V2

2.4.1 Purpose and Threat Model

Without Secure Boot, an attacker with physical flash access can replace the firmware with arbitrary malicious code, modify the bootloader to bypass security checks, or inject persistent backdoors that survive OTA updates. Secure Boot V2 addresses this by establishing a *hardware root of trust*: the boot ROM executes from internal read-only memory and verifies cryptographic signatures before any code from flash is executed.

2.4.2 How Secure Boot V2 Works

Secure Boot V2 uses **asymmetric cryptography** (RSA-3072 or ECDSA, depending on the chosen scheme). At power-on:

1. The boot ROM reads from internal, immutable memory.
2. It reads the bootloader image from flash and verifies its RSA-3072 signature against the *SHA-256 digest of the public key* stored in eFuses.
3. If verification passes, the bootloader runs.
4. The bootloader verifies the application image signature before launching it.
5. If any verification fails, execution halts — the device will not run unverified code.

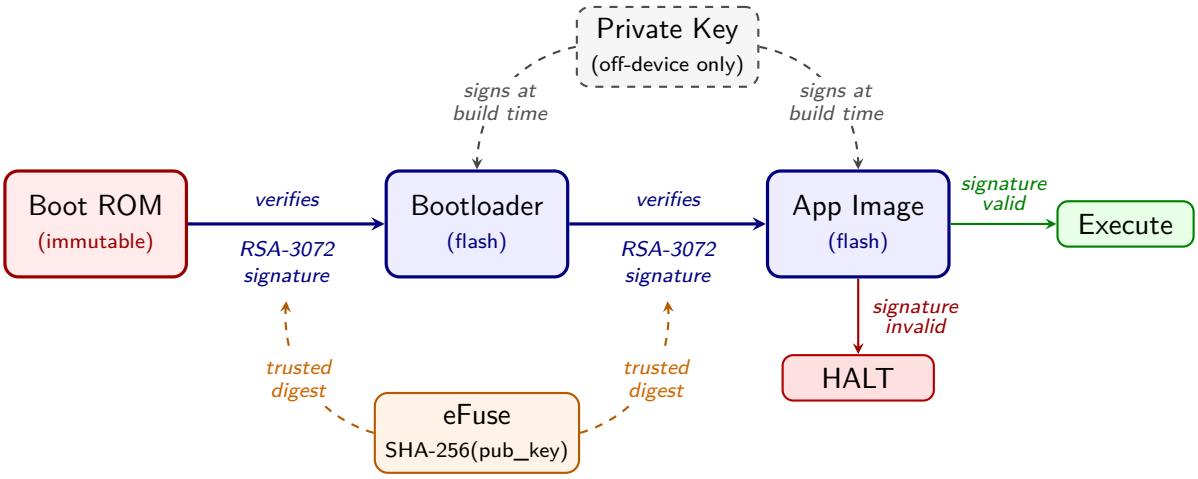


Figure 3: Secure Boot V2 chain of trust

The **private signing key never resides on the device**. Only the SHA-256 digest of the corresponding public key is burned into eFuses during provisioning. An attacker who modifies flash cannot change the expected digest, so the chain of trust remains intact.

2.4.3 Key Generation

```
idf.py secure-generate-signing-key \
    --version 2 \
    --scheme rsa3072 \
    secure_boot_private_key.pem
```

Listing 5: Generating the Secure Boot V2 RSA-3072 signing key

The resulting `.pem` file must never be flashed to the device and must never enter the general development environment. It is the root of all trust.

2.4.4 Manual Signing Workflow for Fleet Management

In production scenarios with multiple devices, a more secure workflow separates build and signing:

1. Disable *Sign binaries during build* in `menuconfig`. The build system produces an unsigned `.bin`.
2. Transfer the unsigned binary to a secure signing station (or HSM) where the private key is stored.
3. Sign with `espsecure.py`:

```
espsecure.py sign_data \
    --version 2 \
    --keyfile secure_boot_private_key.pem \
    --output signed_firmware.bin \
    build/unsigned_firmware.bin
```

Listing 6: Signing a firmware binary with espsecure.py

```
espsecure.py verify_signature \
    --version 2 \
    --keyfile secure_boot_private_key.pem \
    signed_firmware.bin
```

Listing 7: Verifying the signature before deployment

This **"Build Once, Sign Once, Deploy to All"** strategy ensures firmware integrity across large device fleets without exposing the private key.

eFuse Permanence

Once Secure Boot eFuses are burned, the configuration is irreversible. There is no mechanism to re-enable unsigned firmware or replace the trusted public key digest on a production device. The *Allow potentially insecure options* flag must never be enabled in production builds.

2.4.5 Interaction with OTA Updates

When both Secure Boot V2 and OTA are enabled, `esp_https_ota_finish()` automatically verifies the RSA signature of the newly downloaded image before updating the `otadata` partition to point to the new slot. An image that does not carry a valid signature is silently rejected and the previous firmware remains active.

3 Over-the-Air (OTA) Firmware Updates

3.1 System Overview

The OTA system consists of two parts: a **Python HTTPS server** running on a development PC that serves the firmware binary, and an **OTA client** running on the ESP32-S3 that connects, checks the version, and downloads the new image.

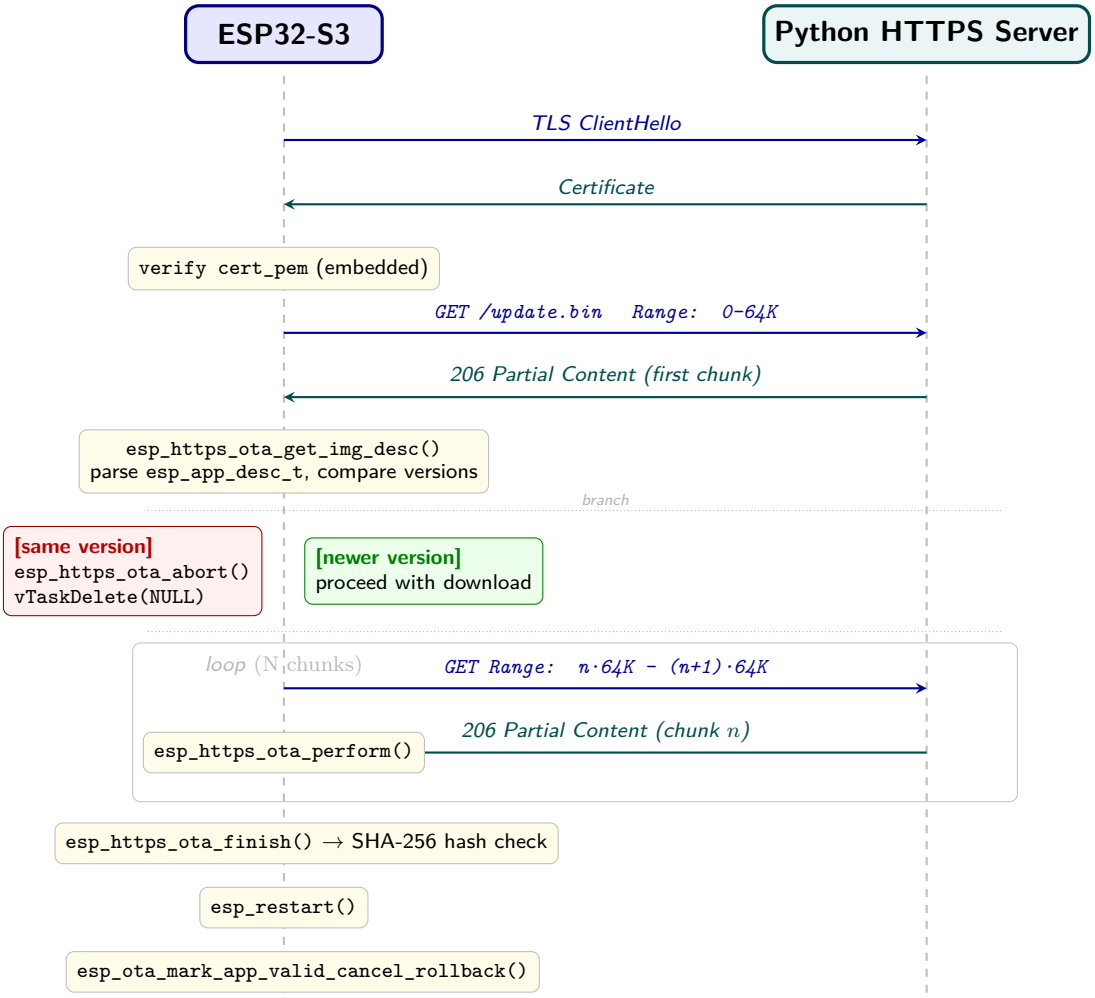


Figure 4: OTA update sequence diagram

The TLS handshake authenticates the server using the certificate embedded in the firmware. Only the first 64 KB chunk is fetched before the version check; if no update is needed the connection is aborted immediately. The loop box represents `N` sequential range requests until the full image has been written to the inactive OTA slot.

3.2 Firmware Server

3.2.1 TLS Certificate

OTA downloads happen over HTTPS to protect the channel against man-in-the-middle attacks. In a development context, a self-signed certificate is sufficient:

```
openssl req -x509 -newkey rsa:2048 \
    -keyout server.key \
    -out server.crt \
    -days 365 -nodes \
    -subj "/CN=OTA-Server"
```

Listing 8: Generating the self-signed TLS certificate

The `server.crt` file is embedded into the firmware at build time (see Section 3.3.2). The `server.key` never leaves the server.

3.2.2 Range-Request Support

The ESP-IDF HTTPS OTA library downloads the firmware in multiple HTTP range requests (chunks up to 64 KB). Python's built-in `SimpleHTTPRequestHandler` does not support `Range` headers, so a custom subclass is required:

```
class RangeHTTPServer(http.server.SimpleHTTPRequestHandler):
    def send_head(self):
        path = self.translate_path(self.path)
        if not os.path.isfile(path):
            return super().send_head()

        f = open(path, 'rb')
        size = os.fstat(f.fileno()).st_size
        range_header = self.headers.get('Range')

        if range_header:
            match = re.match(r'bytes=(\d+)-(\d*)', range_header)
            if match:
                start = int(match.group(1))
                end = int(match.group(2)) if match.group(2)
                                          else size - 1
                self.send_response(206)  # Partial Content
                self.send_header('Content-Range',
                                 f'bytes {start}-{end}/{size}')
                self.send_header('Content-Length',
                                 str(end - start + 1))
                self.end_headers()
                f.seek(start)
                return f
        # Full-file fallback
        self.send_response(200)
        self.send_header('Content-Length', str(size))
        self.end_headers()
        return f
```

Listing 9: Range-capable HTTP handler (firmware_server.py)

When the ESP32-S3 aborts the download early (e.g., after inspecting the version header), the server handles the broken TCP connection gracefully:

```
except (ConnectionResetError, BrokenPipeError):
    print("Client closed connection (version match or abort).")
    return
```

Listing 10: Graceful handling of early client disconnection

3.3 ESP32-S3 Firmware Side

3.3.1 Partition Table

The ESP-IDF OTA mechanism requires a dual-slot partition layout. A single-app table is not sufficient. The `partitions.csv` used in this project is:

```
# Name,      Type,  SubType,  Offset,  Size,     Flags
nvs,         data,  nvs,      0xe000,  16K,
otadata,     data,  ota,      ,        8K,
phy_init,    data,  phy,      ,        4K,
nvs_key,     data,  nvs_keys, ,        4K,       encrypted
ota_0,       app,   ota_0,    ,        1800K,
ota_1,       app,   ota_1,    ,        1800K,
```

Listing 11: partitions.csv — dual-slot OTA layout

The `otadata` partition stores a counter and CRC so the bootloader knows which slot to activate and whether it has been marked valid. Each OTA slot is 1800 KB, yielding a total flash requirement of approximately 4 MB.

3.3.2 CMakeLists.txt and Certificate Embedding

The server certificate is embedded directly into the firmware binary at build time, avoiding any filesystem dependency:

```
idf_component_register(
    SRCS main.c
    INCLUDE_DIRS "."
    REQUIRES nvs_flash bt esp_http_client esp_https_ota
   app_update esp_partition
    PRIV_REQUIRES esp_wifi esp_netif led_strip
)
target_add_binary_data(${COMPONENT_TARGET} "server.crt" TEXT)
```

Listing 12: main/CMakeLists.txt — embedding the TLS certificate

In C, the certificate is accessed through linker-generated symbols:

```
extern const uint8_t server_crt_start[] asm("
   _binary_server_crt_start");
```

Project versioning is set in the top-level `CMakeLists.txt` *before* the `project()` call, so that it populates the `esp_app_desc_t` structure and is accessible to the version comparison logic:

```
set(PROJECT_VER "1.0.0")
project(OTA_update VERSION ${PROJECT_VER})
```

3.3.3 OTA Configuration Structures

The OTA session is configured through two nested structures:

```
// HTTP layer: TLS certificate, timeout, keep-alive
esp_http_client_config_t http_config = {
    .url = "https://192.168.0.128:8070/update.bin",
    .cert_pem = (const char *)server_crt_start,
    .timeout_ms = 10000,
    .keep_alive_enable = true,
    .skip_cert_common_name_check = true, // CN not validated
};

// OTA layer: enable partial (chunked) download
esp_https_ota_config_t ota_config = {
    .http_config = &http_config,
    .partial_http_download = true,        // For version pre-check
    .max_http_request_size = 64 * 1024,   // 64 KB per request
};
```

Listing 13: HTTP and OTA configuration structures

3.3.4 Semantic Version Check and Rollback Prevention

Before downloading the full image, the firmware reads only the first chunk to extract the `esp_app_desc_t` header and compare versions:

```
esp_app_desc_t incoming_app_desc;
err = esp_https_ota_get_img_desc(https_ota_handle, &
   incoming_app_desc);
if (err == ESP_OK) {
    const esp_app_desc_t *running = esp_app_get_description();
    if (!is_newer(running->version, incoming_app_desc.version)) {
        ESP_LOGW("OTA", "Not a newer version. Aborting.");
        esp_https_ota_abort(https_ota_handle); // Closes cleanly
        vTaskDelete(NULL);
        return;
    }
}
```

Listing 14: Version check before committing to the full download

The semantic version comparison function:

```
bool is_newer(const char* running, const char* incoming) {
    int r_major, r_minor, r_patch;
    int i_major, i_minor, i_patch;
    sscanf(running, "%d.%d.%d", &r_major, &r_minor, &r_patch);
    sscanf(incoming, "%d.%d.%d", &i_major, &i_minor, &i_patch);
    if (i_major > r_major) return true;
    if (i_major < r_major) return false;
    if (i_minor > r_minor) return true;
    if (i_minor < r_minor) return false;
    return (i_patch > r_patch);
}
```

Listing 15: Semantic version comparison (MAJOR.MINOR.PATCH)

The function returns `true` *only* if the incoming version is strictly greater. Equal or lower versions are both rejected, simultaneously preventing redundant re-flashing and deliberate rollback attacks.

3.3.5 Download Loop and Finalisation

```
while (1) {
    err = esp_https_ota_perform(https_ota_handle);
    if (err != ESP_ERR_HTTPS_OTA_IN_PROGRESS) break;
    ESP_LOGD("OTA", "Bytes written: %d",
             esp_https_ota_get_image_len_read(https_ota_handle));
}
if (esp_https_ota_finish(https_ota_handle) == ESP_OK) {
    ESP_LOGI("OTA", "OTA upgrade successful! Rebooting...");
    vTaskDelay(pdMS_TO_TICKS(1000));
    esp_restart();
}
```

Listing 16: Chunked download loop

Each call to `esp_https_ota_perform()` downloads one 64 KB chunk and writes it directly to the inactive OTA slot, avoiding large RAM buffers. `esp_https_ota_finish()` verifies the SHA-256 hash (and RSA signature if Secure Boot is active) before updating `otadata`.

3.3.6 Rollback Protection

After rebooting into the new firmware, the application must confirm it is healthy early in `app_main()`:

```
esp_err_t err = esp_ota_mark_app_valid_cancel_rollback();
if (err == ESP_OK) {
    ESP_LOGI("OTA", "App is valid! Rollback cancelled.");
}
```

Listing 17: Rollback cancellation — called early in app_main()

If the application crashes or resets *before* reaching this call, the bootloader automatically reverts to the previous slot on the next power-on — providing a fail-safe against a bricked device.

Table 4: OTA update sequence summary

Step	API Call	Action
1	`esp_https_ota_begin()`	TLS connect, fetch first chunk
2	`esp_https_ota_get_img_desc()`	Read app header, compare versions
3	`esp_https_ota_abort()` (if needed)	Close cleanly if no update needed
4	`esp_https_ota_perform()` × N	Download and write chunks to flash
5	`esp_https_ota_finish()`	Verify hash/signature, update `otadata`
6	`esp_restart()`	Reboot into new firmware
7	`esp_ota_mark_app_valid_cancel_rollback()`	Confirm health, cancel rollback

4 Bluetooth LE Subsystem

4.1 Overview

The Bluetooth subsystem allows a smartphone to send RGB colour values to the ESP32-S3, which drives a WS2812B addressable LED. The firmware exposes a single custom GATT service using the **NimBLE** stack (built into ESP-IDF). The Android companion application is built with Kotlin and Jetpack Compose.

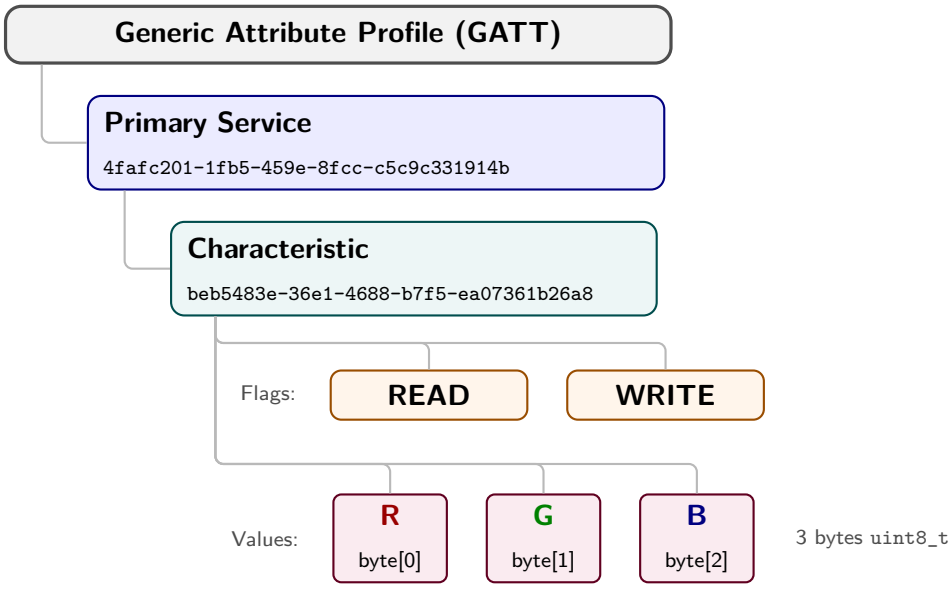


Figure 5: BLE GATT hierarchy exposed by the ESP32-S3

The single characteristic accepts 3-byte write operations `[R, G, B]` and updates the WS2812B LED colour immediately via the `gatt_svr_chr_handler()` callback.

4.2 Firmware — NimBLE GATT Server

4.2.1 Service and Characteristic Definitions

The GATT service tree is declared statically in `main.h`:

```
// Service UUID: 4fafc201-1fb5-459e-8fcc-c5c9c331914b
static const ble_uuid128_t gatt_svr_svc_uuid =
    BLE_UUID128_INIT(0x4b, 0x91, 0x31, 0xc3, 0xc9, 0xc5, 0xcc,
     0x8f, 0x9e, 0x45, 0xb5, 0x1f, 0x01, 0xc2, 0xaf, 0x4f);

// Characteristic UUID: beb5483e-36e1-4688-b7f5-ea07361b26a8
static const ble_uuid128_t gatt_svr_chr_uuid =
    BLE_UUID128_INIT(0xa8, 0x26, 0x1b, 0x36, 0x07, 0xea, 0xf5,
     0xb7, 0x88, 0x46, 0xe1, 0x36, 0x3e, 0x48, 0xb5, 0xbe);

static const struct ble_gatt_svc_def gatt_svr_svcs[] = {
    { .type = BLE_GATT_SVC_TYPE_PRIMARY,
      .uuid = &gatt_svr_svc_uuid.u,
      .characteristics = (struct ble_gatt_chr_def[]) {{
```

```
            .uuid = &gatt_svr_chr_uuid.u,
            .access_cb = gatt_svr_chr_handler,
            .flags = BLE_GATT_CHR_F_READ | BLE_GATT_CHR_F_WRITE,
        }, { 0 }},  // end of characteristics
    }, { 0 },       // end of services
};
```

Listing 18: GATT service and characteristic declaration

4.2.2 GATT Write Handler

When the Android app writes 3 bytes (R, G, B) to the characteristic, the handler extracts them and updates the LED immediately:

```
static int gatt_svr_chr_handler(uint16_t conn_handle, uint16_t
    attr_handle,
                                    struct ble_gatt_access_ctxt *
    ctxt, void *arg) {
    if (ctxt->op == BLE_GATT_ACCESS_OP_WRITE_CHR) {
        uint16_t len = OS_MBUF_PKTLEN(ctxt->om);
        if (len == 3) {
            uint8_t rgb[3];
            ble_hs_mbuf_to_flat(ctxt->om, rgb, 3, NULL);
            update_led_color(rgb[0], rgb[1], rgb[2]);
        }
        return 0;
    }
    return BLE_ATT_ERR_UNLIKELY;
}
```

Listing 19: GATT characteristic write callback

4.2.3 GAP Advertising and Reconnection

The device advertises with its name (`ESP32_RGB_LED`) and automatically restarts advertising on disconnection:

```
static int ble_gap_event(struct ble_gap_event *event, void *arg)
    {
    switch (event->type) {
        case BLE_GAP_EVENT_CONNECT:
            ESP_LOGI(TAG, "BLE Connected");
            break;
        case BLE_GAP_EVENT_DISCONNECT:
            ESP_LOGI(TAG, "BLE Disconnected; restarting
                           advertising");
            ble_app_advertise();  // Immediately re-advertise
            break;
    }
    return 0;
}
```

Listing 20: GAP event handler — restart advertising on disconnect

4.2.4 LED Driver

The WS2812B LED is driven via the ESP-IDF `led_strip` component over the RMT (Remote Control Transceiver) peripheral, which generates the precise timing pulses required by the WS2812B protocol:

```
void init_led_strip() {
    led_strip_config_t strip_config = {
        .strip_gpio_num = LED_STRIP_GPIO,        // GPIO 48
        .max_leds = LED_STRIP_NUM,               // 1 LED
        .color_component_format =
   LED_STRIP_COLOR_COMPONENT_FMT_GRB,
        .led_model = LED_MODEL_WS2812,
    };
    led_strip_rmt_config_t rmt_config = {
        .clk_src = RMT_CLK_SRC_DEFAULT,
        .resolution_hz = 10 * 1000 * 1000,
    };
    ESP_ERROR_CHECK(led_strip_new_rmt_device(&strip_config,
                    &rmt_config, &led_strip));
}

void update_led_color(uint8_t r, uint8_t g, uint8_t b) {
    led_strip_set_pixel(led_strip, 0, r, g, b);
    led_strip_refresh(led_strip);
}
```

Listing 21: LED strip initialisation and colour update

4.3 Android Application — Jetpack Compose

The companion application is a native Android app written in Kotlin with **Jetpack Compose** for the UI. It performs BLE scanning, connection management, and colour command transmission.

4.3.1 Permissions and Manifest

```
<uses-permission android:name="android.permission.BLUETOOTH"
    android:maxSdkVersion="30" />
<uses-permission android:name="android.permission.BLUETOOTH_SCAN"
    android:usesPermissionFlags="neverForLocation" />
<uses-permission android:name="android.permission.
   BLUETOOTH_CONNECT" />
<uses-permission android:name="android.permission.
   ACCESS_FINE_LOCATION" />
```

Listing 22: AndroidManifest.xml — BLE permissions

4.3.2 BLE Manager

The `BleManager` class wraps the Android Bluetooth stack:

```
// UUIDs match the ESP32-S3 firmware exactly
private val SERVICE_UUID = UUID.fromString("4fafc201-1fb5-459e-8
   fcc-c5c9c331914b")
private val CHARACTERISTIC_UUID = UUID.fromString("beb5483e-36e1
   -4688-b7f5-ea07361b26a8")

fun connect(device: BluetoothDevice) {
    bluetoothGatt = device.connectGatt(context, false,
   gattCallback)
}

fun sendColor(red: Int, green: Int, blue: Int) {
    val gatt = bluetoothGatt ?: return
    val characteristic = rgbCharacteristic ?: return
    val data = byteArrayOf(red.toByte(), green.toByte(), blue.
   toByte())

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        gatt.writeCharacteristic(characteristic, data,
                                 BluetoothGattCharacteristic.
   WRITE_TYPE_DEFAULT)
    } else {
        @Suppress("DEPRECATION")
        characteristic.value = data
        @Suppress("DEPRECATION")
        gatt.writeCharacteristic(characteristic)
    }
}
```

Listing 23: BleManager.kt — UUIDs and connection (extract)

The implementation handles both the new Android 13+ (API 33) BLE write API and the legacy deprecated API for older devices, using a version check at runtime.

State is exposed to the UI through Kotlin **StateFlow**:

```
val scannedDevices: StateFlow<List<BluetoothDevice>>
val isConnected: StateFlow<Boolean>
```

4.3.3 UI — Jetpack Compose

The UI is built with Compose and follows a simple state-machine structure:

1. **Permission request**: The app requests `BLUETOOTH_SCAN` and `BLUETOOTH_CONNECT` on Android 12+, or `ACCESS_FINE_LOCATION` on older versions.
2. **Scan screen**: Lists discovered BLE devices; tapping a device connects to it.
3. **Controller screen**: Three sliders (R, G, B, range 0–255) send colour commands on every value change. A live preview square shows the current colour.

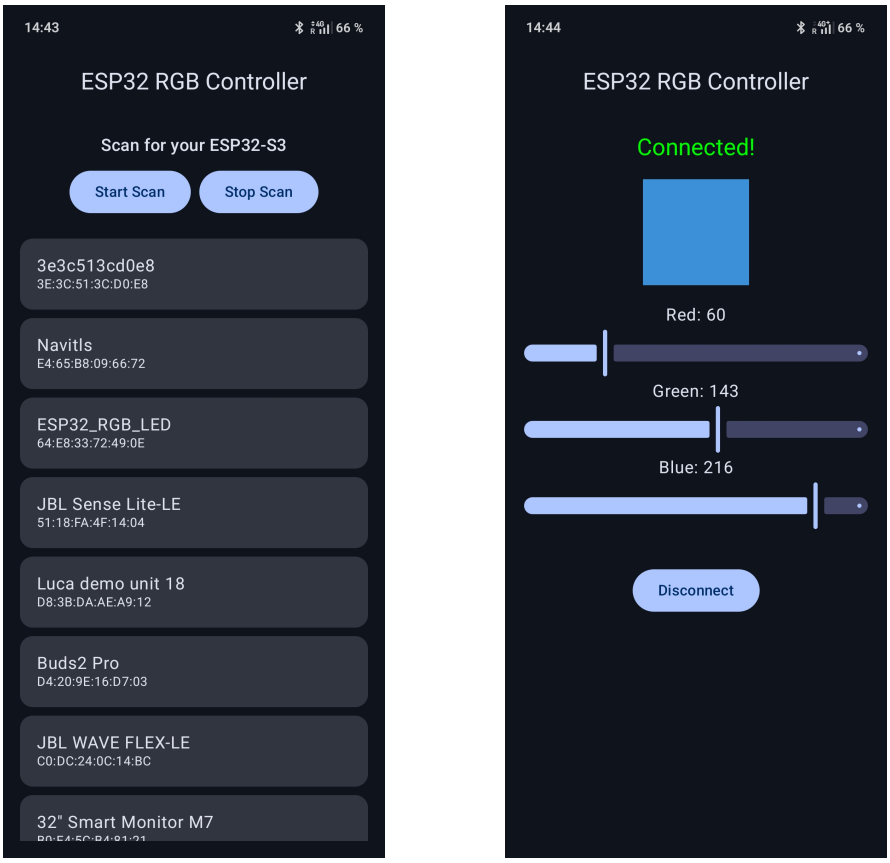


Figure 6: Screenshots of the Android application

```
@Composable
fun ColorPicker(onColorChanged: (Int, Int, Int) -> Unit) {
    var red by remember { mutableFloatStateOf(0f) }
    var green by remember { mutableFloatStateOf(0f) }
    var blue by remember { mutableFloatStateOf(0f) }

    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        // Live colour preview
        Box(modifier = Modifier.size(100.dp)
            .background(Color(red/255f, green/255f, blue/255f)))
        Spacer(modifier = Modifier.height(16.dp))

        Text("Red: ${red.toInt()}")
        Slider(value = red, onValueChange = {
         red = it
         onColorChanged(red.toInt(), green.toInt(), blue.toInt())
        }, valueRange = 0f..255f)
        // (Green and Blue sliders follow the same pattern)
    }
}
```

Listing 24: ColorPicker composable — sliders update the LED in real time

5 Conclusion

This project demonstrates the design and integration of three security mechanisms — Flash Encryption, NVS Encryption, and Secure Boot V2 — alongside a production-grade OTA update system and a BLE-controlled LED application on the ESP32-S3 platform.

Key engineering insights:

- **Layered security is necessary.** Flash encryption alone is insufficient against code-execution attacks; NVS encryption alone is insufficient against physical attacks. The two mechanisms address complementary threat models and are designed to be used together, with flash encryption protecting the NVS encryption keys.
- **Partition table design is as important as enabling encryption.** Misconfiguring the `encrypted` flag — either missing it on sensitive data partitions, or incorrectly applying it to NVS data partitions — silently breaks the intended security model.
- **Rollback protection is a first-class concern.** A partial or corrupted OTA update is potentially more damaging than no update. The dual-partition scheme with `esp_ota_mark_app_valid_cancel_rollback()` provides a hardware-enforced safety net.
- **The build pipeline is part of the security model.** Separating firmware compilation from signing (Secure Boot V2 manual workflow) prevents the private key from ever entering the development environment.

Possible Extensions

- Replace the self-signed OTA certificate with one signed by an internal CA to enable proper hostname validation (`skip_cert_common_name_check = false`).
- Add BLE security (pairing and bonding) to prevent unauthenticated LED control.
- Store WiFi credentials in the encrypted NVS rather than in `menuconfig` to avoid embedding them in the firmware binary.()